Recommender at Scale: Latency, Cost, and Popularity Bias

A Streaming Comparison of Apache Spark vs. Apache Flink on MIND-small News Impressions
Course Project Final Report | Demo Date: 2 November 2026
Authors: Sripada Krishna Sameer Aditya (PES1UG24AM285), Shree Verdhan M (PES1UG24AM265), Shrivamshi Chikkireddi (PES1UG24AM287)

1. Problem Statement & Motivation

Modern real-time news recommendation systems must serve personalized article rankings within strict millisecond-level SLAs while ingesting thousands of concurrent reader impression events. Concurrently, unconstrained point-wise CTR models systematically suffer from severe popularity bias, disproportionately concentrating impression exposure on a handful of viral articles and dominant publishers while starving niche publishers of reader reach. This project benchmarks two dominant open-source distributed streaming frameworks—Apache Spark Structured Streaming (micro-batch) and Apache Flink / PyFlink (event-driven DataStream)—evaluating end-to-end ingestion latency, system saturation boundaries, hardware resource efficiency, and the streaming runtime overhead of an exposure-aware fair re-ranking mechanism.

2. Dataset Provenance & Privacy Guarantees

All reported benchmark metrics and offline ranking evaluations are derived exclusively from the official Microsoft News Dataset (MIND-small) downloaded from Hugging Face (Recommenders/MIND) under the Microsoft Research License Terms. The training corpus comprises 51,282 news articles across 156,965 reader behavior sessions (CTR = 4.0446%); the evaluation set comprises 42,416 news articles across 73,152 validation sessions (CTR = 4.0636%). Archive and extracted file integrity was strictly validated against recorded SHA-256 checksums in data/raw/mind_small.sha256. A MovieLens-25M conversion pipeline was retained strictly as an emergency offline fallback when upstream CDN endpoints failed, and zero MovieLens data feeds any reported number in this study (citing Harper & Konstan 2015).

User Privacy: Raw MIND user identifiers (e.g. U12345) are never logged, exported, or passed into the streaming message bus. The streaming replay producer executes a cryptographically salted SHA-256 hash (using MIND_USER_HASH_SALT) on all user IDs, ensuring strict one-way pseudonymization across all Kafka topics and container environments.

3. Architecture & Dual-Engine Streaming Pipeline

The system pipeline is orchestrated via Docker Compose, comprising four interconnected subsystems: (1) A deterministic Replay Producer that streams serialized JSON impression events from dev behaviors into Kafka at target rates from 100 to 4,000 ev/s; (2) An Apache Kafka cluster configured with 2 input topic partitions (impressions) and 2 output partitions (recommendations) operating with server-side message.timestamp.type=LogAppendTime; (3) Stream Processing Engines executing identical Python business logic (recsys/scorer.py and recsys/reranker.py); and (4) A Benchmark Metric Consumer that evaluates end-to-end latency, throughput parity, message loss, and queue drain.

Engine Implementations: • Spark Structured Streaming (spark/pass_through.py): Ingests micro-batches with configurable triggers. We evaluate both the default trigger (eager micro-batch execution upon input arrival) and spark-1s (ProcessingTime=1 second). • PyFlink DataStream API (flink/pass_through.py): Executes true pipelined stream processing using Python UDFs. We evaluate single-operator parallelism (flink-p1), dual-operator parallelism (flink-p2), and an optimized bundle buffer configuration (flink-p1-tuned with python.fn-execution.bundle.time tuned to 50 ms).

4. Recommender Model & Fair Re-Ranking Algorithm

Base Scorer: For each candidate article i in impression session u, the base score combines Laplace-smoothed CTR (alpha=10.0) with historical category affinity:

$$\text{score}(u, i) = (\text{clicks}_i + 10 * \text{global_ctr}) / (\text{impressions}_i + 10) + \beta * \text{affinity}(u, \text{category}_i)$$

Fair Re-Ranker: MIND lacks native publisher metadata; to study exposure inequality, items are partitioned across 10 synthetic publisher groups using six distinct mappings (3 uniform random: random_101..303; 3 category-correlated: category_404..606). Target publisher shares are set to empirical candidate shares in the training split. The re-ranker greedily selects the top-5 served articles to penalize over-exposed groups:

$$i^* = \text{argmax}_{i \in C} [\ \text{score}(u, i) - \lambda * \max(0, \text{curr_share}(g_i) - \text{target_share}(g_i))\ ]$$

State is maintained across impressions using an exponential discount factor (gamma=0.999), dynamically rewarding under-represented publishers.

5. Experimental Methodology & Metric Definitions

• Single-Variable Capacity Probe (bench/probe.py): Sweeps rates {100, 250, 500, 1000, 2000, 4000} ev/s across 7 engine configurations for 30s steps to identify saturation thresholds.
• 42-Run Benchmark Matrix (bench/run.py): 7 configurations (spark-default, spark-1s, flink-p1, flink-p1-tuned, flink-p2 at lambda=0; plus spark-default-rerank and flink-p1-rerank at lambda=0.6) evaluated across rates

{1000, 2000} ev/s with 3 independent replications. Runs use 30s warm-up and 60s measurement windows.
• End-to-End Latency Metric: Measured strictly as (Kafka LogAppendTime of recommendations - Producer Timestamp of impression). This captures total transport, queueing, serialization, and engine processing delay.
• Resource & Cost Tracking: Evaluated using Linux cgroup v2 usage_usec (CPU seconds) and memory.peak. Cost figures are ASSUMED and illustrative, calculated using illustrative pricing parameters ($0.0001/vCPU-s, $0.00001/GB-s) from config/model.yaml.

6. Systems Performance & Saturation Results

Table 1 summarizes the empirical matrix results across 42 benchmark runs. Below saturation (1,000 ev/s), all engines maintain zero message loss and match the target throughput. However, their latency and resource profiles diverge substantially:

Configuration	Rate (ev/s)	p50 Lat (ms)	p95 Lat (ms)	Throughput (ev/s)	Backlog @ 60s	CPU / 1k ev (s)	Assumed Cost/1M
spark-default	1000	291.7 ± 14.6	501.7 ± 41.9	1005.0	231	1.693	$0.1781
spark-default	2000	6171.3 ± 638.7	11696.7 ± 1159.3	1732.1	18,459	0.617	$0.0660
spark-1s	1000	920.0 ± 5.3	1350.3 ± 8.1	1003.8	815	1.028	$0.1102
spark-1s	2000	6571.7 ± 138.3	12489.0 ± 492.7	1721.8	19,402	0.607	$0.0649
flink-p1	1000	1008.3 ± 5.1	1459.0 ± 2.6	1000.5	1,147	0.782	$0.0916
flink-p1	2000	5281.3 ± 570.6	8849.7 ± 638.5	1748.7	15,913	0.549	$0.0612
flink-p1-tuned	1000	216.3 ± 16.3	333.0 ± 33.2	999.5	185	0.824	$0.0953
flink-p1-tuned	2000	5296.7 ± 308.8	9219.0 ± 389.9	1728.1	16,716	0.550	$0.0613
flink-p2	1000	1776.3 ± 3.5	2663.0 ± 4.6	1010.7	1,358	0.816	$0.0964
flink-p2	2000	4440.7 ± 509.5	7414.3 ± 382.5	1813.5	13,406	0.598	$0.0673
spark-default-rerank	1000	304.7 ± 13.2	506.3 ± 39.9	1006.9	254	1.705	$0.1791
spark-default-rerank	2000	7018.3 ± 533.2	13394.3 ± 1018.7	1698.6	20,435	0.614	$0.0656
flink-p1-rerank	1000	1016.7 ± 10.6	1469.0 ± 9.5	995.3	1,160	0.786	$0.0915
flink-p1-rerank	2000	6075.3 ± 626.6	10685.0 ± 753.6	1698.0	18,864	0.547	$0.0611

Key Systems Findings:
1. Flink's Bundle Buffer Floor: In default Flink (flink-p1), PyFlink's Python worker buffers output records before emitting to Kafka, imposing a latency floor of ~1,008 ms at 1,000 ev/s. Tuning python.fn-execution.bundle.time to 50 ms (flink-p1-tuned) collapses p50 latency to 216.3 ms, outperforming Spark's default trigger (291.7 ms).
2. Spark Trigger Tradeoff: At 1,000 ev/s, introducing a 1s trigger (spark-1s) increases median latency from 291.7 ms to 920.0 ms, but reduces CPU consumption by 39.3% (from 1.693 to 1.028 CPU-s per 1k events) by amortizing micro-batch driver coordination overhead.
3. Saturation Collapse at 2,000 ev/s: Both single-node engines hit maximum throughput limits (~1,720–1,748 ev/s). At 2,000 ev/s, ingestion exceeds processing capacity, causing massive Kafka backlog accumulation (18,459 events in Spark; 15,913 in Flink-p1) and p50 latencies exceeding 5 seconds.

7. Fair Re-Ranking Runtime Overhead

Table 2 measures the performance overhead of enabling stateful fair re-ranking (lambda=0.6) compared to the unconstrained base scorer.

Engine	Rate (ev/s)	Metric	Base Scorer	With Reranker (λ=0.6)	% Change
spark-default	1000	latency_p50_kafka	291.67 ± 14.57	304.67 ± 13.20	+4.46%
spark-default	1000	latency_p95_kafka	501.67 ± 41.86	506.33 ± 39.93	+0.93%
spark-default	1000	output_throughput	1005.00 ± 0.36	1006.91 ± 0.51	+0.19%
spark-default	1000	backlog_at_window_end	231.00 ± 60.23	253.67 ± 18.18	+9.81%
spark-default	1000	cpu_per_1k_events	1.69 ± 0.04	1.71 ± 0.04	+0.74%
spark-default	2000	latency_p50_kafka	6171.33 ± 638.67	7018.33 ± 533.23	+13.72%

spark-default	2000	latency_p95_kafka	11696.67 ± 1159.33	13394.33 ± 1018.75	+14.51%
spark-default	2000	output_throughput	1732.06 ± 24.34	1698.56 ± 8.28	-1.93%
spark-default	2000	backlog_at_window_end	18459.33 ± 1758.99	20435.33 ± 806.48	+10.70%
spark-default	2000	cpu_per_1k_events	0.62 ± 0.00	0.61 ± 0.00	-0.54%
flink-p1	1000	latency_p50_kafka	1008.33 ± 5.13	1016.67 ± 10.60	+0.83%
flink-p1	1000	latency_p95_kafka	1459.00 ± 2.65	1469.00 ± 9.54	+0.69%
flink-p1	1000	output_throughput	1000.50 ± 8.58	995.29 ± 1.53	-0.52%
flink-p1	1000	backlog_at_window_end	1147.33 ± 277.15	1160.33 ± 247.99	+1.13%
flink-p1	1000	cpu_per_1k_events	0.78 ± 0.01	0.79 ± 0.01	+0.60%
flink-p1	2000	latency_p50_kafka	5281.33 ± 570.59	6075.33 ± 626.56	+15.03%
flink-p1	2000	latency_p95_kafka	8849.67 ± 638.48	10685.00 ± 753.59	+20.74%
flink-p1	2000	output_throughput	1748.68 ± 11.10	1697.98 ± 26.26	-2.90%
flink-p1	2000	backlog_at_window_end	15913.00 ± 505.75	18863.67 ± 1410.21	+18.54%
flink-p1	2000	cpu_per_1k_events	0.55 ± 0.00	0.55 ± 0.00	-0.33%

Analysis of Re-ranking Costs: At 1,000 ev/s, greedy re-ranking adds minimal overhead (Spark p50 latency increases +4.46%; Flink-p1 p50 increases +0.83%), and CPU increases by less than 0.8%. However, at saturation (2,000 ev/s), the additional computation exacerbates queueing delay, causing Spark p50 latency to jump +13.72% and Flink backlog to increase by +18.54%.

8. Popularity Bias Mitigation & Bootstrap Confidence Intervals

Offline analysis across the 6 synthetic publisher mappings reveals a distinct Pareto frontier between publisher exposure equality and recommendation utility. Table 3 presents 1,000-sample bootstrap 95% confidence intervals (resampling impression sessions) for category-correlated and random mappings.

Mapping	Lambda (λ)	Metric	Point Estimate	Bootstrap Mean	95% Low CI	95% High CI
category_404	0.0	exposure_l1	1.0128	1.0129	1.0096	1.0162
category_404	0.0	ndcg_at_5_served	0.3073	0.3073	0.3049	0.3097
category_404	5.0	exposure_l1	0.6021	0.6021	0.5986	0.6057
category_404	5.0	ndcg_at_5_served	0.2972	0.2972	0.2948	0.2999
category_404	0.0_to_5.0	ndcg_at_5_served_difference	0.0101	0.0101	0.0084	0.0118
random_101	0.0	exposure_l1	1.0305	1.0306	1.0272	1.0335
random_101	0.0	ndcg_at_5_served	0.3073	0.3073	0.3049	0.3097
random_101	5.0	exposure_l1	0.5783	0.5785	0.5750	0.5822
random_101	5.0	ndcg_at_5_served	0.2974	0.2974	0.2948	0.2999
random_101	0.0_to_5.0	ndcg_at_5_served_difference	0.0099	0.0099	0.0082	0.0115

Bias Findings: Increasing lambda from 0.0 to 5.0 reduces publisher Exposure L1 distance by 39.98% to 43.89% across all 6 synthetic publisher mappings. In contrast, reader recommendation utility (served nDCG@5) degrades by only 1.18% to 4.35% (a drop of 0.0036 to 0.0134 in absolute nDCG), demonstrating that aggressive exposure mitigation can be achieved with modest utility penalties.

Statistical Rigor & Methodology:

• Resampling Protocol: 1,000 bootstrap resamples at the impression session level (n = 73,152), preserving user click histories and intra-impression candidate ranking dependencies.

• Single-Class AUC Convention: In news sessions where a reader clicked all candidates or clicked none, AUC is assigned a neutral baseline value of 0.5, preventing division-by-zero errors and session-dropping selection bias.

• Candidate vs. Served Metrics: Candidate pool AUC (0.6178) and candidate nDCG@10 (0.3699) evaluate global offline model ranking and are invariant across lambda because re-ranking only adjusts the served top-5 list (served nDCG@5).

• Paired Utility Differences: In individual 95% CIs for served nDCG@5, marginal intervals exhibit slight overlap due to inter-impression session variance. However, within-impression paired bootstrap analysis (ndcg_at_5_served_difference) proves that the utility drop is statistically significant across all mappings, as the 95% paired confidence intervals strictly exclude zero.

9. Resource Cost Analysis (ASSUMED)

> NOTE: ALL COST FIGURES IN THIS STUDY ARE ASSUMED AND ILLUSTRATIVE. Unit prices ($0.0001 per vCPU-second and $0.00001 per GB-second of memory) are derived from config/model.yaml as analytical cost models, not cloud service quotes.

At 1,000 ev/s, Spark Default incurs an assumed cost of $0.1781 per 1M events, whereas Flink-p1 incurs $0.0916 and Flink-p1-tuned incurs $0.0953 per 1M events. Flink achieves ~46% lower cost per million events under steady-state load due to lower Python-JVM serialization overhead. At 2,000 ev/s, cost per event drops for all engines because CPU resources saturate while input queueing absorbs excess load.

10. Limitations & Future Work

1. Synthetic Publisher Groupings: MIND lacks real-world publisher metadata; our groups are analytical controls. Proportional exposure to synthetic groups does not measure journalistic fairness.
2. State Partitioning & Checkpointing: In this single-node benchmark harness, re-ranking exposure state is kept in driver memory without external checkpointing or partitioning by publisher key; multi-partition scaling causes partitioned state divergence.
3. Static Feature Tables: Candidate click counters are frozen from train behaviors; production news systems require rolling online CTR streaming updates.
4. Exact Event Validation & Container Limits: In streaming runs with live Kafka producers, event IDs are randomly generated UUID4s without an external expected ID registry; historical runs were validated on distinct counts and schema completeness rather than exact pre-assigned ID sets. Docker CPU and memory resource limits represent declarative Compose deployment specifications rather than statically verified host-enforced cgroups.
5. Future Work: Evaluating distributed multi-broker Kafka clusters with RocksDB state backend in Flink and RocksDB stateful operators in Spark Structured Streaming.